The `smc2fc` Framework

A Practical Reference for Sequential Monte Carlo^2
Filtering and Stochastic Optimal Control

smc2fc development team

May 6, 2026

Abstract

This document is a practical reference for the `smc2fc` package — a JAX implementation of Sequential Monte Carlo^2 for Bayesian filtering of state-space models with intractable likelihoods, paired with a control-as-inference receding-horizon controller. It is deliberately model-independent: the underlying SDE model and observation likelihood are abstractions supplied by the user, and nothing in this document depends on the specifics of any particular application.

The intended reader is a research engineer who needs to use, extend, or tune the framework. The treatment is technical where it must be (the Bayesian filtering recursion, the unbiased likelihood property, the adaptive tempering schedule) and pedagogic where it helps (the control-as-inference framing, the receding-horizon principle, the soft-barrier feasibility argument). Two kinds of empirical findings are documented: the JAX-native compile-once architecture (which speeds up inference by an order of magnitude) and the result that NUTS, despite being the textbook upgrade from HMC, is roughly 100× slower than HMC inside this particular nested control flow. Both are written so future agents do not repeat the dead-end.

The document deliberately omits speculation about algorithmic alternatives the framework does not currently implement. The one open direction it does cover — improving the soft-HMC controller's wall-clock without sacrificing control quality — is the only bottleneck the authors have hit empirically.

Contents

1 Overview **4**
- 1.1 Scope and non-scope . . . 4
- 1.2 Architecture in one diagram . . . 4
- 1.3 Reading order . . . 4
- 1.4 Conventions . . . 5
- 1.5 What this document does not promise . . . 5

2 The generic state-space setting **6**
- 2.1 The hidden Markov model in continuous time . . . 6
- 2.2 The two abstractions the framework consumes . . . 6
- 2.3 What the framework does *not* require . . . 7
- 2.4 Notational summary . . . 7

3 The bootstrap particle filter **7**
- 3.1 The Bayesian filtering recursion . . . 7
- 3.2 Sequential importance resampling (SIR) . . . 8
- 3.3 Liu–West shrinkage . . . 8
- 3.4 The unbiased marginal likelihood . . . 9
- 3.5 Implementation pointer . . . 9

4 Optimal-transport rescue inside the filter **9**
4.1 When SIR plus Liu–West is not enough . . . 9
4.2 The OT step itself — low-rank Sinkhorn . . . 10
4.3 The full OT-rescue API . . . 11
4.4 Implementation pointer . . . 11
4.5 Practical takeaway . . . 11

5 SMC^2 over parameters: tempered SMC + HMC **11**
5.1 The tempered posterior . . . 12
5.2 Adaptive tempering: choosing $\lambda_k - \lambda_{k-1}$ . . . 12
5.3 The HMC move step . . . 12
5.4 Adaptive diagonal mass matrix . . . 12
5.5 The full SMC^2 loop . . . 13
5.6 Implementation pointer . . . 13

6 JAX-native SMC: once-per-run compilation **13**
6.1 The compile-cost problem with the BlackJAX-wrapped backend . . . 14
6.2 The fix . . . 14
6.3 Empirical speedup . . . 14
6.4 Failed experiment: NUTS as a drop-in replacement . . . 15
6.5 How to switch backends . . . 15
6.6 Implementation pointers . . . 15
6.7 Practical takeaway . . . 16

7 Gaussian bridge for warm-starts **16**
7.1 The Gaussian bridge in practice . . . 16
7.2 The Schrödinger–Föllmer / Bures–Wasserstein variant . . . 16
7.3 Implementation pointer . . . 17

8 Control as inference **17**
8.1 The cost functional . . . 17
8.2 Schedule decoder . . . 17
8.3 Tempered control posterior . . . 18
8.4 Hard-indicator vs soft-sigmoid chance-constrained variants . . . 18
8.5 Implementation pointer . . . 19

9 Receding-horizon model-predictive control **19**
9.1 Finite-horizon stochastic optimal control . . . 19
9.2 The receding-horizon principle . . . 20
9.3 Stochastic + output-feedback variants . . . 20
9.4 The closed-loop pipeline — generic algorithm . . . 20
9.5 Driver pattern: plant, filter, controller . . . 21
9.6 Recursive feasibility and soft barriers . . . 22
9.7 The flat-cost-surface argument . . . 22

10 Configuration and knobs **22**
10.1 Outer SMC^2 (the parameter loop) . . . 22
10.2 Inner particle filter . . . 23
10.3 OT-rescue (Section 4) . . . 23
10.4 Bridge (Section 7) . . . 24
10.5 Other config dataclasses . . . 24
10.6 Tuning workflow . . . 24

11 Future work: optimising the soft-HMC controller **25**
11.1 The trade-off as observed today . . . 25
11.2 The tunable knobs and what each one costs . . . 25
11.3 Open directions . . . 26
11.4 What's not on this list . . . 27

A Code-API quick reference **28**
A.1 `smc2fc/core/` . . . 28
A.2 `smc2fc/filtering/` . . . 29
A.3 `smc2fc/control/` . . . 29
A.4 `smc2fc/simulator/` . . . 30
A.5 `smc2fc/transforms/` and top-level . . . 30

1 Overview

The `smc2fc` package implements two coupled algorithms: a Sequential Monte Carlo^2 (SMC^2) Bayesian filter for nonlinear, non-Gaussian state-space models with intractable marginal likelihoods, and a stochastic-optimal-control receding-horizon controller built on the same machinery. The two share the same outer-loop tempered SMC and the same per-particle differentiable likelihood evaluator, applied to different log-densities: the Bayes posterior in the filter case, the control-as-inference posterior in the controller case.

This document describes what is implemented today, with enough mathematical context to read the source files without an external textbook open in parallel.

1.1 Scope and non-scope

In scope. The framework code under `smc2fc/`: the core SMC^2 engine (`smc2fc/core/`), the inner particle filter (`smc2fc/filtering/`), the control-spec abstraction and the tempered-control loop (`smc2fc/control/`). All mathematical content here applies to any user model that conforms to the framework's two interfaces (Section 2).

Out of scope. Specific user models. `smc2fc` is designed to be coupled to a user-supplied SDE plus observation model; the FSA family of physiological models in the sibling `FSA_model_dev` repository is one such model and is documented separately. Nothing in this document depends on the FSA model's drift, state space, or parameters.

1.2 Architecture in one diagram

The framework has four layers, depicted in Figure 1. From inside out:

1. **Inner particle filter.** For a fixed parameter vector $\boldsymbol{\theta}_{\text{dyn}}$, runs an SDE-driven bootstrap particle filter over the latent state. Returns an unbiased estimator of the marginal observation likelihood $p(y_{1:T} \mid \boldsymbol{\theta}_{\text{dyn}})$. Implementation: `smc2fc/filtering/gk_dpf_v3_lite.py`.

2. **Outer SMC^2 over parameters.** Runs adaptive tempered SMC over the parameter vector $\boldsymbol{\theta}_{\text{dyn}}$, calling the inner filter to evaluate likelihoods and using HMC moves between tempering steps to rejuvenate the particle cloud. Implementations: `smc2fc/core/tempered_smc.py` (BlackJAX wrapper, kept as a reference) and `smc2fc/core/jax_native_smc.py` (the production compile-once backend; see Section 6).

3. **Controller.** Reuses the outer SMC^2 engine, but with the data-likelihood replaced by a control-as-inference cost-likelihood $\exp(-\beta J(\boldsymbol{\theta}_{\text{ctrl}}))$ over a finite- dimensional schedule parameter $\boldsymbol{\theta}_{\text{ctrl}}$. Returns a posterior cloud over schedules; the posterior mean is the applied control. Implementation: `smc2fc/control/`.

4. **MPC driver.** Orchestrates the receding-horizon loop: for each replan time, advance the plant under the previous stride's control, refresh the parameter posterior on the new observation window, then call the controller for the next stride. The driver is user code; the framework provides the ingredients.

1.3 Reading order

The document is structured to be readable straight through, but two shortcuts are useful.

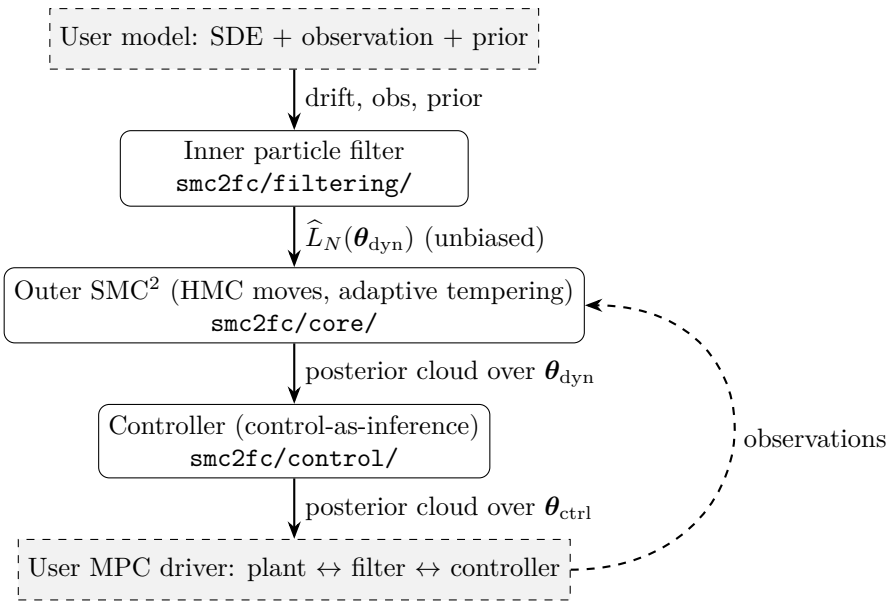

Figure 1: The four layers of the `smc2fc` framework. Solid blocks are framework code; dashed blocks are user code. The dashed arrow on the right closes the receding-horizon loop: each stride's observations become the next stride's likelihood input.

Mathematician's path. If you want the mathematical anchors first, read in this order: Section 2 (the HMM setup), Section 3 (Bayesian filtering recursion and the unbiased likelihood property), Section 5 (the tempered posterior), Section 8 (control-as-inference), Section 9 (receding-horizon principle).

Engineer's path. If you want to use or modify the code, read in this order: Section 2 (the two interfaces you must conform to), Section 6 (the production compile-once backend and how to switch between it and the BlackJAX wrapper), Section 10 (every knob in `SMCConfig`, what each one does and how to tune it), Section 8 (the two chance-constrained cost variants and which one to use), Appendix A (one-line API lookup).

1.4 Conventions

- $\boldsymbol{\theta}_{\text{dyn}}$ denotes the dynamics-model parameter vector (inferred from data); $\boldsymbol{\theta}_{\text{ctrl}}$ denotes the schedule parameter vector (chosen by the controller). Both live in $\mathbb{R}^d$ for some user-defined d.
- X_n is the latent state at step n, Y_n the observation, Φ_n (or generic U_n) the exogenous control input at the same step.
- N is the number of *state* particles (inner PF); M is the number of *parameter* particles (outer SMC^2). Code uses `n_pf_particles` and `n_smc_particles` respectively.
- File:line citations are checked against the current source at the time of writing. If the line numbers drift (e.g. after a refactor), the function name remains the canonical reference.

1.5 What this document does not promise

The framework is research code, not a textbook implementation. It has been tuned against a small number of model classes (the FSA physiological models, the SWAT hydrological model). Hyperparameter defaults are reasonable starting points for similar classes; for genuinely different

problems they will need re-tuning, and Section 10 is the diagnostic guide for doing so. Where the framework has hit empirical limits, those limits are documented in Sections 6–8 and Section 11, not glossed.

2 The generic state-space setting

This section pins down the abstractions `smc2fc` expects from the user's model. Everything in subsequent sections depends only on these two interfaces.

2.1 The hidden Markov model in continuous time

The framework assumes a continuous-time stochastic system observed at discrete times. The latent state $X(t) \in \mathbb{R}^{n_x}$ evolves under a controlled Itô stochastic differential equation

$$\mathrm{d}X(t) \;=\; b(X(t),\, U(t),\, \boldsymbol{\theta}_{\mathrm{dyn}})\,\mathrm{d}t \;+\; \sigma(X(t),\, \boldsymbol{\theta}_{\mathrm{dyn}})\,\mathrm{d}W(t), \tag{1}$$

with drift $b : \mathbb{R}^{n_x} \times \mathbb{R}^{n_u} \times \mathbb{R}^d \to \mathbb{R}^{n_x}$, diffusion $\sigma : \mathbb{R}^{n_x} \times \mathbb{R}^d \to \mathbb{R}^{n_x \times n_x}$ (typically diagonal in framework usage), Brownian motion $W(t)$, an exogenous control input $U(t) \in \mathbb{R}^{n_u}$, and a static parameter vector $\boldsymbol{\theta}_{\mathrm{dyn}} \in \mathbb{R}^d$. Time is discretised on a uniform grid $t_n = n\Delta t$, $n = 0, 1, 2, \ldots$, and equation (1) is integrated step-by-step by Euler–Maruyama:

$$X_{n+1} \;=\; X_n \;+\; b(X_n, U_n, \boldsymbol{\theta}_{\mathrm{dyn}})\Delta t \;+\; \sigma(X_n, \boldsymbol{\theta}_{\mathrm{dyn}})\sqrt{\Delta t}\,\xi_{n+1}, \qquad \xi_{n+1} \sim \mathcal{N}(0, I). \tag{2}$$

At each time step $n \geq 1$ the user obtains an observation $Y_n \in \mathbb{R}^{n_y}$ via the user-supplied likelihood

$$Y_n \mid X_n \;\sim\; p(\,\cdot \mid X_n,\, C_n,\, \boldsymbol{\theta}_{\mathrm{dyn}}), \tag{3}$$

where C_n is any per-step exogenous regressor (e.g. a circadian forcing, a calendar feature) that the user wants to expose to the observation model. The framework supports Gaussian, Bernoulli, and mixed channels in $p(Y_n \mid X_n, \cdot)$ — all log-likelihoods are summed across channels and any per-bin presence mask supplied by the user.

The framework does not place semantic constraints on X_n, U_n, or Y_n. It only requires that the drift, diffusion, observation likelihood, and prior $p(\boldsymbol{\theta}_{\mathrm{dyn}})$ are all evaluable and JAX-traceable.

2.2 The two abstractions the framework consumes

(A) The forward simulator: `SDEModel`. A frozen dataclass that bundles the drift, diffusion, observation samplers, and per-state physical bounds. Used by the user's `StepwisePlant` (Section 9) for closed-loop ground-truth simulation, and by the test infrastructure for synthetic data generation. See `smc2fc/simulator/sde_model.py`.

(B) The inference target: `EstimationModel`. A frozen dataclass that bundles the propagate function, the diffusion function, the observation log-weight function, and configuration of the parameter prior. The framework's outer SMC^2 engine reads only from this. See `smc2fc/estimation_model.py`.

The user supplies one of each. The two are typically thin wrappers around the same underlying drift and observation code; the split exists because the simulator side wants numpy-friendly per-step samplers suitable for a Python loop, while the inference side wants pure-JAX callables suitable for batching over many particles inside `jax.vmap`.

2.3 What the framework does *not* require

- *No closed-form transition density.* The framework only needs to be able to *sample* from $p(X_n \mid X_{n-1}, U_n, \boldsymbol{\theta}_{\text{dyn}})$, never to evaluate it pointwise. This is what makes it applicable to nonlinear SDEs.
- *No tractable marginal likelihood.* $p(Y_{1:T} \mid \boldsymbol{\theta}_{\text{dyn}})$ never has to be evaluated exactly; the bootstrap particle filter (Section 3) provides an unbiased estimator that the outer SMC^2 then operates on.
- *No Gaussianity.* Neither the transition kernel nor the observation likelihood needs to be Gaussian. The HMC moves in the outer SMC^2 act on the parameter vector $\boldsymbol{\theta}_{\text{dyn}}$, not on the latent state.
- *No separation principle.* The framework does not assume the controller can be designed assuming a perfect state estimate. The receding-horizon loop of Section 9 explicitly threads an SMC^2 posterior through the controller without separating estimation from control.

2.4 Notational summary

Symbol	Meaning	Code
$X_n \in \mathbb{R}^{n_x}$	Latent state at step n	`state` (6D in FSA, 35D in SWAT)
$Y_n \in \mathbb{R}^{n_y}$	Observation at step n	per-channel `obs_value` arrays
U_n, Φ_n	Exogenous control input at step n	`Phi` (sometimes `U`)
C_n	Per-step exogenous regressor	`C`
$\boldsymbol{\theta}_{\text{dyn}} \in \mathbb{R}^d$	Static parameter vector (inferred)	`u` or `theta_dyn`
$\boldsymbol{\theta}_{\text{ctrl}}$	Control-schedule parameter vector	`theta_ctrl`
N	Number of inner-PF state particles	`n_pf_particles`
M	Number of outer-SMC parameter particles	`n_smc_particles`
λ	Tempering parameter, $0 \to 1$	`lam` or `tempering_param`
$\widehat{L}_N(\boldsymbol{\theta}_{\text{dyn}})$	Unbiased likelihood estimator	`log_density(u, ...)`

3 The bootstrap particle filter

This section presents the inner Bayesian filtering layer. For a fixed parameter vector $\boldsymbol{\theta}_{\text{dyn}}$, the particle filter estimates the sequence of conditional state distributions $\pi_n(\mathrm{d}x_n) := p(\mathrm{d}x_n \mid Y_{1:n}, \boldsymbol{\theta}_{\text{dyn}})$ and, as a by-product, returns an unbiased estimator of the marginal observation likelihood $p(Y_{1:T} \mid \boldsymbol{\theta}_{\text{dyn}})$. That estimator is the hook that makes the outer SMC^2 of Section 5 possible.

3.1 The Bayesian filtering recursion

Under the SDE–observation model of Section 2, with $\boldsymbol{\theta}_{\text{dyn}}$ fixed, the filtering distribution satisfies the prediction–update recursion

$$\pi_{n|n-1}(\mathrm{d}x_n) = \int p(\mathrm{d}x_n \mid x_{n-1}, U_n, \boldsymbol{\theta}_{\text{dyn}})\, \pi_{n-1}(\mathrm{d}x_{n-1}), \tag{4}$$

$$\pi_n(\mathrm{d}x_n) \propto p(Y_n \mid x_n, C_n, \boldsymbol{\theta}_{\text{dyn}})\, \pi_{n|n-1}(\mathrm{d}x_n). \tag{5}$$

Equations (4)–(5) have closed form only when both densities are linear-Gaussian (the Kalman case). In the general nonlinear non-Gaussian setting they do not, and a Monte Carlo approximation is used.

3.2 Sequential importance resampling (SIR)

The bootstrap particle filter (Gordon, Salmond and Smith, 1993; Doucet and Johansen, 2011) approximates π_n by a weighted empirical measure

$$\pi_n^N(\mathrm{d}x_n) \;=\; \sum_{i=1}^{N} W_n^{(i)}\,\delta_{X_n^{(i)}}(\mathrm{d}x_n), \qquad \sum_{i=1}^{N} W_n^{(i)} = 1, \tag{6}$$

on a population of N *state particles* $\{X_n^{(i)}\}$. Each particle carries a normalised importance weight $W_n^{(i)}$. The recursion has three substeps per step: propagate, reweight, resample. This is summarised in Algorithm 1.

Algorithm 1 Bootstrap SIR for a fixed $\boldsymbol{\theta}_{\mathrm{dyn}}$

1: Sample $X_0^{(i)} \sim p(\mathrm{d}x_0 \mid \boldsymbol{\theta}_{\mathrm{dyn}})$ for $i = 1, \ldots, N$; set $W_0^{(i)} = 1/N$.
2: **for** $n = 1, 2, \ldots$ **do**
3: **Propagate.** For each i, draw $X_n^{(i)} \sim p(\cdot \mid X_{n-1}^{(i)}, U_n, \boldsymbol{\theta}_{\mathrm{dyn}})$ by one Euler–Maruyama step of (2).
4: **Reweight.** Set $\tilde{W}_n^{(i)} = W_{n-1}^{(i)}\, p(Y_n \mid X_n^{(i)}, C_n, \boldsymbol{\theta}_{\mathrm{dyn}})$ and normalise $W_n^{(i)} = \tilde{W}_n^{(i)} / \sum_j \tilde{W}_n^{(j)}$.
5: **ESS check.** Compute $N_{\mathrm{eff}} = 1/\sum_i (W_n^{(i)})^2$. If $N_{\mathrm{eff}} < N_{\mathrm{thr}}$, systematically resample $\{X_n^{(i)}\}$ by $\{W_n^{(i)}\}$ and reset $W_n^{(i)} \leftarrow 1/N$.
6: **end for**

Systematic resampling. The framework uses the *systematic* variant: with $u_0 \sim \mathcal{U}[0,1)$, the resampled indices are obtained by walking the cumulative weight $C(j) = \sum_{i \le j} W_n^{(i)}$ at the offsets $u_k = (k + u_0)/N$ for $k = 0, \ldots, N-1$. Systematic resampling is $O(N)$, has lower Monte Carlo variance than multinomial resampling, and is deterministic given u_0. Implementation: `blackjax.smc.resampling.systematic`, called from `smc2fc/filtering/gk_dpf_v3_lite.py`.

3.3 Liu–West shrinkage

The textbook bootstrap filter of Algorithm 1 is, in production, run *coupled* to the outer SMC^2 over $\boldsymbol{\theta}_{\mathrm{dyn}}$ (Section 5). At each tempering step the parameter cloud is jittered by HMC, and the inner filter is re-run. Without intervention, this coupled-sampler scheme suffers from *parameter–state cloud collapse*: the inner state cloud's variance shrinks too quickly because the resampling is not aware of the parameter degree of freedom.

The Liu–West (2001) shrinkage correction addresses this. Let h be a Silverman bandwidth on the stochastic state subspace and define the shrinkage coefficient $a = \sqrt{1 - h^2}$. After resampling, each particle is moved toward the weighted state mean:

$$X_n^{(i)} \;\leftarrow\; a\, X_n^{(i)} \;+\; (1-a)\,\mu_W, \qquad \mu_W = \sum_i W_n^{(i)}\, X_n^{(i)}. \tag{7}$$

The choice $a = \sqrt{1 - h^2}$ ensures the empirical variance of the shrunk cloud equals the unshrunk variance plus the kernel-bandwidth inflation, preserving the second moment exactly. In production the bandwidth itself is *ESS-scaled*: when the cloud is healthy ($N_{\mathrm{eff}} \approx N$) the bandwidth is near zero and the shrinkage is a near-identity; when the cloud collapses ($N_{\mathrm{eff}} \to 1$) the bandwidth equals the Silverman default and full shrinkage fires.

The continuous mapping $N_{\mathrm{eff}} \to h \to$ (shrink amount) replaces a discontinuous threshold and is critical for gradient-based outer-loop kernels: see the lengthy comment block in `smc2fc/filtering/_gk_kernel.py:38--56` for the engineering history (an MCLMC step-size tuner regression that was caused by the discontinuity and fixed by ESS-scaled bandwidth).

3.4 The unbiased marginal likelihood

A key property of the bootstrap filter, which makes the SMC^2 algorithm of Section 5 possible, is that it produces an unbiased estimator of the marginal observation likelihood. With the unnormalised weights $\tilde{W}_n^{(i)}$ from line 4 of Algorithm 1, define

$$\widehat{L}_N(\boldsymbol{\theta}_{\text{dyn}}) \;:=\; \prod_{n=1}^{T}\Bigg(\frac{1}{N}\sum_{i=1}^{N}\tilde{W}_n^{(i)}\Bigg). \tag{8}$$

Proposition 3.1 (Unbiasedness; Del Moral, 2004)**.** For any $N \geq 1$ and any $T \geq 1$, $\mathbb{E}[\widehat{L}_N(\boldsymbol{\theta}_{\text{dyn}})] = p(Y_{1:T} \mid \boldsymbol{\theta}_{\text{dyn}})$.

A proof is in Del Moral (2004, Theorem 7.4.2). For the present notes the significance is operational: it allows us to plug $\widehat{L}_N(\boldsymbol{\theta}_{\text{dyn}})$ in place of the (intractable) exact likelihood $p(Y_{1:T} \mid \boldsymbol{\theta}_{\text{dyn}})$ inside any sampler over $\boldsymbol{\theta}_{\text{dyn}}$ and obtain a procedure that targets exactly the right posterior. This is precisely what SMC^2 does (Section 5).

3.5 Implementation pointer

The compile-once entry point is `smc2fc.filtering.gk_dpf_v3_lite.make_gk_dpf_v3_lite_log_density_compileonce`. Given an `EstimationModel` and inner-PF / OT-rescue hyperparameters, it returns a JIT-compiled callable `log_density(u, grid_obs, fixed_init_state, w_start, key0)` that runs the full bootstrap filter for a given parameter vector $u = \boldsymbol{\theta}_{\text{dyn}}$ and observation buffer, returning $\log \widehat{L}_N(u)$ plus a small state dict.

The Gaussian-kernel resampling primitives that implement the ESS-scaled Liu–West shrinkage of Section 3.3 live in `smc2fc/filtering/_gk_kernel.py` (functions `smooth_resample_ess_scaled_lw` and friends). The optional optimal-transport rescue invoked inside the same scan is the subject of Section 4.

4 Optimal-transport rescue inside the filter

The Liu–West shrinkage of Section 3.3 keeps the inner state cloud well-behaved most of the time. Occasionally — under a particularly informative observation, or when the parameter cloud visits a low-likelihood region during HMC exploration — the inner filter's effective sample size collapses despite the smoothed resampling. The framework includes a low-rank Sinkhorn optimal-transport "rescue" kernel that fires when this happens, blending a transport plan into the systematic resample by a sigmoid weight on the ESS ratio. This section explains what it does and why it has the form it does.

4.1 When SIR plus Liu–West is not enough

After the Liu–West shrinkage step of Section 3.3, the inner state cloud's effective sample size N_{eff} is monitored once per bin. If it falls below an OT-rescue threshold (default $N_{\text{eff}}/N < 0.05$, controlled by `ot_ess_frac`), the cloud is judged degenerate enough that systematic resampling will produce too many duplicates and the filter's ability to track the state will be lost. The rescue is a soft, sigmoid-blended OT resampling rather than a hard branch.

The blending weight is

$$w_{\text{OT}} \;=\; w_{\text{OT}}^{\max} \;\cdot\; \text{sigmoid}\Bigg(\frac{\text{ess_thr} - N_{\text{eff}}/N}{\text{ess_temp}}\Bigg), \tag{9}$$

with $w_{\text{OT}}^{\max}$ a small cap (default 0.01, so even at full saturation only 1% of the resample comes from OT), ess_thr = `ot_ess_frac`, and ess_temp = `ot_temperature`. The output of the

resampling step at each bin is then

$$X^{\text{out}} \;=\; (1 - w_{\text{OT}})\, X^{\text{sys+LW}} \;+\; w_{\text{OT}}\, X^{\text{OT}}, \tag{10}$$

i.e. a convex combination of the standard systematic+Liu–West output and the OT-resampled output. Because both inputs are $\mathbb{P}$-preserving and the weight is in $[0, w_{\text{OT}}^{\max}]$, the combination is also $\mathbb{P}$-preserving.

The continuous-blend formulation matters for the same reason ESS-scaled bandwidth matters in Section 3.3: the outer SMC^2 HMC kernel (Section 5) needs the inner-PF log-likelihood to be a smooth function of $\boldsymbol{\theta}_{\text{dyn}}$. A discontinuous ESS-threshold branch would create gradient discontinuities at every particle that crosses the threshold, sabotaging the HMC step-size adaptation.

4.2 The OT step itself — low-rank Sinkhorn

The OT-rescue kernel takes the current particle cloud $\{X^{(i)}\}_{i=1}^{N}$ with weights $\{W^{(i)}\}$ and produces a new cloud whose empirical distribution approximates the same distribution as systematic resampling, but via a transport plan rather than a multinomial draw. The transport plan's smoothness with respect to $\boldsymbol{\theta}_{\text{dyn}}$ is what makes the gradient go through.

Step 1 — low-rank kernel factorisation. Pick r anchor particles by index (`anchor_idx`, sampled uniformly without replacement) and form the Gaussian kernel between all N particles and these r anchors:

$$K_{NR}^{(i,j)} \;=\; \exp\!\left(-\frac{\|X^{(i)} - X^{(\text{anchor}\, j)}\|^2}{2\,\epsilon}\right), \qquad i = 1, \ldots, N;\; j = 1, \ldots, r, \tag{11}$$

column-normalised for stability. This is the Nyström approximation of the full $N \times N$ Gaussian kernel by $K_{NR}K_{NR}^{\top}$. Cost: $O(Nr)$ in time and memory. Implementation: `smc2fc/filtering/transport_kernel.py:compute_kernel_factor`.

The low-rank form is essential because the full $N \times N$ kernel, combined with Sinkhorn's $O(N^2)$ matrix-vector products, would exhaust GPU memory at the rank-$\sim$400 cloud sizes the framework uses. Empirical default $r = 5$ (`ot_rank`); tested up to $r = 50$ on tighter problems.

Step 2 — Sinkhorn iterations in scaling form. Solve the entropy-regularised optimal transport problem with target marginals a (current importance weights) and b (uniform weights): find scalings $u, v \in \mathbb{R}_{>0}^{N}$ such that

$$\operatorname{diag}(u)\, K_{\text{approx}} \operatorname{diag}(v) \quad \text{has marginals} \quad (a,\, b), \qquad K_{\text{approx}} := K_{NR}K_{NR}^{\top}. \tag{12}$$

Sinkhorn alternates the row-marginal and column-marginal projections

$$u^{(\ell+1)} = a/(K_{\text{approx}} v^{(\ell)}), \qquad v^{(\ell+1)} = b/(K_{\text{approx}}^{\top} u^{(\ell+1)}), \tag{13}$$

each iteration costing $O(Nr)$ via $K_{\text{approx}} v = K_{NR}(K_{NR}^{\top} v)$. Default $\ell_{\max} = 2$ (`ot_n_iter`). Implementation: `smc2fc/filtering/sinkhorn.py:sinkhorn_scalings`.

A subtle but important detail: the iteration is implemented via `jax.lax.fori_loop`, not `lax.scan`. The reason (documented at the top of `sinkhorn.py`) is that `lax.scan` inside a particle-filter scan branch causes XLA to unroll all inner iterations, leading to compilation-time blow-up. `fori_loop` keeps the inner iteration as a real on-device loop.

Step 3 — barycentric projection. The transport plan $\Gamma = \mathrm{diag}(u)\, K_{\mathrm{approx}}\, \mathrm{diag}(v)$ implicitly defines a coupling between source and target particles. The new particle positions are computed as the barycentric projection

$$X_i^{\mathrm{OT}} \;=\; \frac{u_i\,[K_{\mathrm{approx}}(v \odot X)]_i}{u_i\,[K_{\mathrm{approx}}\, v]_i}. \tag{14}$$

This is a convex combination of source positions, so it preserves any $[0,1]^d$ box constraints automatically (no logit / sigmoid needed for state-bounded models). Implementation: `smc2fc/filtering/project.py:barycentric_projection`.

4.3 The full OT-rescue API

The three steps above are combined in `smc2fc/filtering/resample.py:ot_resample_lr`, which is the only function the inner PF actually calls. Its signature:

```
ot_resample_lr(particles, log_weights, rng_key, stochastic_indices,
               epsilon=0.5, n_iter=10, rank=50) -> (N, n_states)
```

The `stochastic_indices` argument tells the OT kernel which state coordinates are subject to noise (and thus need rescuing); deterministic state components pass through unchanged. The function is fully differentiable through `jax.grad` — this is the key practical feature, distinguishing it from systematic resampling which has zero gradient with respect to particle positions.

4.4 Implementation pointer

- `smc2fc/filtering/transport_kernel.py` (∼75 lines): Nyström kernel factorisation, factor matvec.
- `smc2fc/filtering/sinkhorn.py` (∼90 lines): low-rank Sinkhorn iterations via `fori_loop`.
- `smc2fc/filtering/project.py` (∼45 lines): barycentric projection.
- `smc2fc/filtering/resample.py` (∼100 lines): the public `ot_resample_lr` API.
- `smc2fc/filtering/gk_dpf_v3_lite.py:50` imports `ot_resample_lr`; the sigmoid-blend interpolation between systematic+LW and OT outputs is in the inner-PF body.

4.5 Practical takeaway

The OT rescue is a *safety net*, not the main rejuvenation mechanism. With the default `ot_max_weight = 0.01`, the OT contribution to the final particle cloud is tiny in normal operation and only becomes appreciable when the cloud is genuinely degenerate. Tuning advice: leave the OT-rescue defaults alone unless you see inner-PF log-likelihood NaNs or sustained $N_{\mathrm{eff}} < 5\%$ across many bins. If you do, the lever is `ot_max_weight` (raise to 0.05–0.10) and `ot_n_iter` (raise to 5–10).

5 SMC^2 over parameters: tempered SMC + HMC

The bootstrap filter of Section 3 assumes the static parameter vector $\boldsymbol{\theta}_{\mathrm{dyn}}$ is known. In practice $\boldsymbol{\theta}_{\mathrm{dyn}}$ is itself the principal object of inference, with a prior $p(\boldsymbol{\theta}_{\mathrm{dyn}})$ and posterior

$$p(\boldsymbol{\theta}_{\mathrm{dyn}} \mid Y_{1:T}) \;\propto\; p(\boldsymbol{\theta}_{\mathrm{dyn}})\, p(Y_{1:T} \mid \boldsymbol{\theta}_{\mathrm{dyn}}). \tag{15}$$

A direct sampler is unavailable because $p(Y_{1:T} \mid \boldsymbol{\theta}_{\mathrm{dyn}})$ has no closed form. The SMC^2 algorithm of Chopin, Jacob and Papaspiliopoulos (2013) replaces this likelihood by the unbiased estimator $\widehat{L}_N(\boldsymbol{\theta}_{\mathrm{dyn}})$ from (8) and runs adaptive tempered SMC over the parameter cloud.

5.1 The tempered posterior

For a schedule $0 = \lambda_0 < \lambda_1 < \cdots < \lambda_K = 1$ define the sequence of tempered targets

$$\pi_\lambda(\boldsymbol{\theta}_{\text{dyn}}) \;\propto\; p(\boldsymbol{\theta}_{\text{dyn}})^{1-\lambda}\, \widehat{L}_N(\boldsymbol{\theta}_{\text{dyn}})^{\lambda}, \qquad \lambda \in [0,1]. \tag{16}$$

At $\lambda = 0$ the target is the prior; at $\lambda = 1$ it is, in expectation, the posterior. The interpolation in between bridges the two in a way that keeps the importance weights well-behaved. Sampling from $\pi_{\lambda_K} = \pi_1$ is the goal; the intermediate π_{λ_k} are stepping stones.

5.2 Adaptive tempering: choosing $\lambda_k - \lambda_{k-1}$

A population of M *parameter particles* $\{\boldsymbol{\theta}_{\text{dyn}}^{(m)}\}_{m=1,\ldots,M}$ (default $M = 256$, `n_smc_particles`) is propagated through the schedule. At each step the unnormalised tempering weight is

$$\omega_k^{(m)} \;\propto\; \left(\frac{\widehat{L}_N(\boldsymbol{\theta}_{\text{dyn}}^{(m)})}{\widehat{L}_N(\boldsymbol{\theta}_{\text{dyn}}^{(m)})_{\text{prev}}} \right)^{\lambda_k - \lambda_{k-1}}, \tag{17}$$

and an effective sample size $M_{\text{eff}} = (\sum_m \omega_k^{(m)})^2 / \sum_m (\omega_k^{(m)})^2$ is monitored. The interval $\lambda_k - \lambda_{k-1}$ is chosen *adaptively*, by bisection, to drive M_{eff}/M to a prescribed target (`target_ess_frac`, default 0.5): too large and the cloud collapses; too small and tempering wastes work. The bisection is capped at `max_lambda_inc` (default 0.05 cold-start, 0.10 bridge) so a single step cannot leap to $\lambda = 1$ in one go.

The on-device implementation of this bisection is `smc2fc/core/jax_native_smc.py:_solve_delta_for_ess`; it runs inside a `lax.scan` of fixed depth (30 bisection steps) so the full tempering loop is one compiled XLA graph. This is part of the JAX-native compile-once architecture covered in Section 6.

5.3 The HMC move step

After resampling at temperature λ_k, the parameter cloud is *rejuvenated* by Hamiltonian Monte Carlo moves targeting π_{λ_k}. Without these moves the cloud loses diversity at every resample step, eventually collapsing onto a single value.

Each move is a leapfrog-integrated trajectory of length `hmc_num_leapfrog` (default 8) at step size `hmc_step_size` (default 0.025), accepted/rejected by the Metropolis ratio. The framework runs `num_mcmc_steps` such moves per tempering level (default 5 for cold start, 3 for bridge warm-start). Implementation: `smc2fc/core/jax_native_smc.py:_hmc_step_chain`, which is `jax.vmap`'d across particles.

5.4 Adaptive diagonal mass matrix

HMC's per-dimension step size is set jointly by `hmc_step_size` and the inverse mass matrix M^{-1}, which acts as a preconditioner. A poorly-chosen mass matrix collapses the acceptance rate to zero very quickly during tempering — for the kinds of high-dimensional posteriors the framework targets (35-D SWAT, 37-D FSA-v5), an identity mass matrix collapses HMC at $\lambda \approx 0.3$.

The framework re-estimates a *diagonal* mass matrix at each tempering level from the current parameter cloud:

$$M_{ii}^{-1} \;=\; \text{Var}_m([\boldsymbol{\theta}_{\text{dyn}}^{(m)}]_i) \;+\; \epsilon, \qquad i = 1, \ldots, d. \tag{18}$$

Implementation: `smc2fc/core/mass_matrix.py:estimate_mass_matrix` (20 lines, deliberately minimal). The regularisation $\epsilon = 10^{-4}$ floors the variance to prevent numerical issues for parameters that have collapsed to a delta.

A full (non-diagonal) mass matrix would in principle be a better preconditioner, but on the parameter scales the framework targets the empirical covariance is too noisy at $M = 256$ particles to invert stably. The diagonal version is a deliberate engineering choice; see the comment block in `mass_matrix.py` for the empirical history.

5.5 The full SMC^2 loop

Algorithm 2 stitches the pieces together.

Algorithm 2 Adaptive-tempered SMC^2 over $\boldsymbol{\theta}_{\text{dyn}}$

1: Sample $\boldsymbol{\theta}_{\text{dyn}}^{(m)} \sim p(\boldsymbol{\theta}_{\text{dyn}})$ for $m = 1, \ldots, M$; set $\lambda_0 = 0$.
2: For each m, run the inner bootstrap filter (Algorithm 1) with $\boldsymbol{\theta}_{\text{dyn}}^{(m)}$ to obtain $\widehat{L}_N(\boldsymbol{\theta}_{\text{dyn}}^{(m)})$.
3: **while** $\lambda_k < 1$ **do**
4: **Adaptive λ.** Solve for the smallest increment $\delta \in (0, 1 - \lambda_k]$ such that the tempering ESS ratio with weights $\omega^{(m)} \propto \widehat{L}_N(\boldsymbol{\theta}_{\text{dyn}}^{(m)})^\delta$ equals the target. Cap at `max_lambda_inc`. Set $\lambda_{k+1} = \lambda_k + \delta$.
5: **Reweight & resample.** Compute weights $\omega^{(m)}$, normalise, systematically resample.
6: **Re-estimate mass matrix.** $M_{ii}^{-1} = \text{Var}_m([\boldsymbol{\theta}_{\text{dyn}}^{(m)}]_i) + \epsilon$ from the resampled cloud.
7: **HMC moves.** Run `num_mcmc_steps` HMC trajectories of length `hmc_num_leapfrog` at step size `hmc_step_size`, targeting $\pi_{\lambda_{k+1}}$ on each $\boldsymbol{\theta}_{\text{dyn}}^{(m)}$. Re-evaluate $\widehat{L}_N(\boldsymbol{\theta}_{\text{dyn}}^{(m)})$ for any moved particle by re-running the inner filter.
8: $k \leftarrow k + 1$.
9: **end while**

5.6 Implementation pointer

Two backends implement Algorithm 2:

- `smc2fc/core/tempered_smc.py` — BlackJAX-wrapped version. Easier to read, slower in production because of a per-window JIT recompilation cost.
- `smc2fc/core/jax_native_smc.py` — production compile-once version. Same algorithm, same outputs, but the whole tempering loop is one XLA graph that compiles once per process and is reused across all subsequent windows.

The two are interchangeable from the caller's perspective. Section 6 explains why the JAX-native version exists, what it costs to maintain, and what it returns at the cost of a few hundred extra lines of careful JAX engineering.

6 JAX-native SMC: once-per-run compilation

This section covers what is, in the authors' view, the framework's single most consequential engineering decision: the JAX-native re-implementation of the tempered-SMC outer loop in `smc2fc/core/jax_native_smc.py`, eliminating the per-window JIT recompilation cost of the BlackJAX-wrapped baseline and reducing the end-to-end runtime of a multi-window inference run by roughly an order of magnitude.

It also documents a failed experiment — replacing HMC with NUTS (`smc2fc/core/jax_native_smc_nuts.py`) — so that future contributors do not repeat it.

6.1 The compile-cost problem with the BlackJAX-wrapped backend

The reference implementation `smc2fc/core/tempered_smc.py` is a thin wrapper around BlackJAX's `blackjax.smc.tempered.build_kernel`. BlackJAX bakes the log-likelihood function into the closure of the returned kernel. That is fine for a single run; the issue is that in the rolling-window setting (Section 9) the log-likelihood closes over per- window observation data, so a new kernel is built per window. Every fresh kernel triggers a new `jax.jit` cache entry, and every new cache entry costs a full HLO recompile — on the model classes the framework targets, in the order of ~ 15 seconds per window.

For a 27-window benchmark that recompilation dominates the total wall-clock; the actual sampling work is only a small fraction of the visible cost. This is a structural problem of the BlackJAX API, not of BlackJAX's HMC kernel itself — the kernel is fine; the issue is that it is rebuilt every window.

6.2 The fix

The fix has three parts, all inside `smc2fc/core/jax_native_smc.py`:

(a) Module-level HMC kernel. The HMC kernel itself is built once at module import:

```
HMC_KERNEL = hmc.build_kernel()  # smc2fc/core/jax_native_smc.py:53
HMC_INIT   = hmc.init
```

These are module-level constants, not per-call closures. The kernel's trace cache key depends only on the shape and dtype of its arguments, not on their values. Same shape/dtype across windows $\Rightarrow$ one compile per process.

(b) Pytree-Partial wrapping of per-window callables. The per-window log-density and the prior log-density are passed in as `jax.tree_util.Partial`-wrapped callables. Partial is a JAX-recognised pytree, so its bound-arg *shapes/dtypes* enter the trace cache key but its bound-arg *values* do not. Across windows the data values change but the data shapes don't, so the cache hits (`jax_native_smc.py:121--130`).

(c) On-device tempering loop via `lax.while_loop`. The full adaptive-tempering loop, including the ESS bisection that chooses each $\delta\lambda$, is implemented inside a single `lax.while_loop` (`jax_native_smc.py:252`). The ESS bisection is itself a fixed-iteration `lax.scan` (`jax_native_smc.py:_solve_delta_for_ess`, 30 bisection steps). There is no Python-side iteration, no Python-side branching, no Python-side ESS solver. The whole thing is one XLA graph.

The combined effect: the first window's call pays the JIT compile cost (~ 1.6 seconds for the cached HMC kernel); every subsequent window reuses the compiled artefact and pays only the sampling cost (~ 1.1 seconds per window in the benchmark).

6.3 Empirical speedup

On a synthetic benchmark target with 256 parameter particles and 20 tempering steps:

Stage	HMC backend (jax-native)
JIT compile + first run	1.63 s
Cached run (subsequent windows)	1.09 s per window

For the same benchmark, the BlackJAX-wrapped backend pays roughly 15 seconds of recompilation per window (figures will vary by hardware and HLO cache state); the JAX-native backend amortises this to a one-shot 1.6 s and is then bounded only by the cost of sampling. The net speedup at 27 windows is approximately one order of magnitude.

6.4 Failed experiment: NUTS as a drop-in replacement

The textbook upgrade from HMC is the No-U-Turn Sampler (NUTS), which adapts the trajectory length per step based on a U-turn diagnostic. A JAX-native NUTS variant of `jax_native_smc.py` was implemented in `smc2fc/core/jax_native_smc_nuts.py` (only the kernel differs — everything else is structurally identical). Both kernels were benchmarked side-by-side on the same synthetic target inside the same conda environment.

Stage	HMC	NUTS
JIT compile + first run	1.63 s	102.03 s
Cached run (per window)	1.09 s	94.23 s

The NUTS version is approximately *60× slower at compile time and* 85× *slower per cached window.*

The cause is structural, not parametric. NUTS dynamically determines the trajectory length using an internal `while_loop` (the U-turn check). When this dynamic-control-flow loop is placed inside a `jax.vmap` (over the 256 parameter particles) and an outer `lax.while_loop` (the tempering schedule), JAX is forced to either unroll the dynamic loop — which produces an enormous XLA graph — or compile a deeply nested control-flow structure that the GPU executes inefficiently. Either way the cost compounds across the two outer loops.

The trajectory-length expansion is also unbounded in the worst case. "5 NUTS steps per particle per tempering level" (matching the HMC default `num_mcmc_steps = 5`) translates into $5 \times 8 = 40$ gradient evaluations under HMC, but anywhere from $5 \times 1 = 5$ to $5 \times 1024 = 5120$ evaluations under NUTS depending on the maximum tree depth.

Conclusion. HMC remains the production rejuvenation kernel. The NUTS implementation file `smc2fc/core/jax_native_smc_nuts.py` is preserved as a documented dead-end — not deleted, so the next contributor who has the same "why don't we just use NUTS?" idea can read this section, see the benchmark, and not repeat the experiment.

6.5 How to switch backends

In user code, the backends are interchangeable at the import line:

```python
# Production: JAX-native, compile-once, HMC.
from smc2fc.core.jax_native_smc import (
    run_smc_window_native, run_smc_window_bridge_native,
)

# Reference: BlackJAX wrapper, slower, easier to read.
from smc2fc.core.tempered_smc import (
    run_smc_window, run_smc_window_bridge,
)

# Documented dead-end (do not use; see Section ref{sec:nuts-failed}).
# from smc2fc.core.jax_native_smc_nuts import (
#     run_smc_window_native, run_smc_window_bridge_native,
# )
```

6.6 Implementation pointers

- `smc2fc/core/jax_native_smc.py` (∼ 430 lines) — production HMC backend.
- `smc2fc/core/jax_native_smc_nuts.py` (∼ 430 lines) — the dead-end NUTS variant. Read the file's header docstring before considering it.

- `smc2fc/core/mass_matrix.py` (20 lines) — the diagonal mass-matrix re-estimation discussed in Section 5.4.
- `smc2fc/core/tempered_smc.py` ($\sim$ 370 lines) — BlackJAX-wrapped reference backend.

6.7 Practical takeaway

For any new application the framework targets, default to `jax_native_smc.py`. The BlackJAX wrapper is useful only as a reference when debugging an unfamiliar log-density (its slower trace cycle gives you cleaner stack traces). The NUTS variant should not be used.

7 Gaussian bridge for warm-starts

When data arrive sequentially and the framework is run in a rolling- window mode (Section 9), the first window is a cold start: the parameter cloud is sampled from the prior and tempered all the way to the posterior. Doing the same on every subsequent window would be wasteful — successive windows' posteriors are typically close to each other, so a *warm-start* from the previous window's posterior is the right thing to do. This section documents the warm-start mechanism the framework actually uses; it is short by design.

7.1 The Gaussian bridge in practice

The framework's default warm-start mechanism (`bridge_type = "gaussian"`) does the simplest thing that works:

1. Fit a single Gaussian $q_0 = \mathcal{N}(m_0, \Sigma_0)$ to the previous window's posterior cloud by importance-weighted moment match (with Liu–West shrinkage on Σ_0 to reduce its noise).
2. Run a single inner-PF evaluation on the new window's data per previous-cloud particle to get an importance-weighted moment match $q_1 = \mathcal{N}(m_1, \Sigma_1)$ of the new posterior.
3. Initialise the next tempering chain from the blended Gaussian $\mathcal{N}(m_b, \Sigma_b)$ with $m_b = (1-\rho)m_0 + \rho m_1$, Σ_b a similar interpolation, and $\rho \in [0, 1]$ a bias toward the new evidence (default $\rho = 0.7$ in `sf_blend`, though the field's value is only used by the SF variant; see below).
4. Run the outer SMC^2 tempering loop from this initialisation, with a slightly relaxed schedule (`max_lambda_inc_bridge = 0.10`, double the cold- start cap) and fewer HMC moves per level (`num_mcmc_steps_bridge = 3`, vs 5 for cold start).

The net effect is that each warm-start window typically converges in a fraction of the cold-start tempering steps, with a smaller HMC budget per step. On a 27-window benchmark this is the difference between a manageable run and an impractical one.

7.2 The Schrödinger–Föllmer / Bures–Wasserstein variant

`smc2fc/core/sf_bridge.py` also implements an alternative warm-start mechanism based on the closed-form Schrödinger–Föllmer bridge between Gaussians, which traces the Bures–Wasserstein geodesic between q_0 and q_1 in the manifold of positive-definite covariance matrices. Activated by `bridge_type = "schrodinger_follmer"` (with `sf_blend = 0.5` for the BW midpoint), it has the appealing property that its midpoint covariance is *larger* than either endpoint's when the endpoints differ — useful in principle for shedding accumulated bias from previous windows.

Empirical assessment. On the model classes the framework has been tested against, the SF/BW variant has *not* produced measurably better posteriors than the simple Gaussian bridge above. The added geometric complexity — matrix square roots, optional entropic regularisation, optional info-aware blending keyed off a local FIM — is not paying for itself. The plain Gaussian bridge is therefore the production default and the recommended setting for new applications.

The SF/BW machinery is preserved in `sf_bridge.py` for two reasons: (a) it may earn its keep on a future, larger-covariance- mismatch problem; (b) several knobs (`sf_info_aware`, `sf_info_lambda_thresh_quantile`, `sf_info_blend_temperature`) are exposed in `SMCConfig` for users who want to experiment. Section 10 summarises them in the configuration reference.

7.3 Implementation pointer

- `smc2fc/core/sf_bridge.py` — the bridge implementation. The Gaussian-bridge path uses a moment-match helper at the top of the file; the SF/BW path uses `bw_geodesic`, `bures_wasserstein_map`, and the larger `fit_sf_base` pipeline.
- `smc2fc/core/jax_native_smc.py:run_smc_window_bridge_native` — the rolling-window entry point that consumes the bridge output and runs the relaxed tempering schedule. Note that this entry point currently only supports the SF/BW variant (the Gaussian bridge is in the BlackJAX-wrapped backend `run_smc_window_bridge`, in `smc2fc/core/tempered_smc.py`).

8 Control as inference

Sections 3–7 treat the dynamics parameters $\boldsymbol{\theta}_{\text{dyn}}$ as the unknown and the schedule U as given. This section reverses the perspective: $\boldsymbol{\theta}_{\text{dyn}}$ is held fixed at its posterior mean (or sampled from the posterior cloud), and the schedule itself becomes the unknown. The framework solves the resulting stochastic- optimal-control problem by re-using the SMC^2 engine of Section 5, with the data-likelihood replaced by an exponential of the negative cost.

8.1 The cost functional

For a fixed dynamics-parameter vector $\boldsymbol{\theta}_{\text{dyn}}$, an initial state x_0, and a candidate schedule $U : [0, T] \to \mathbb{R}^{n_u}$, define the trajectory cost

$$J(U) \;=\; \mathbb{E}_{X|U}\left[\int_0^T \ell(X(t), U(t))\,\mathrm{d}t \;+\; V_f(X(T))\right], \tag{19}$$

where the expectation is over the law of X under the SDE (1) driven by U, ℓ is a stage cost, and V_f is a terminal cost. The framework treats ℓ and V_f as user-supplied callables; common choices include negative running reward, soft state barriers, and effort-quadratic penalties. Concrete examples are given in Section 8.4.

8.2 Schedule decoder

The schedule U is parameterised in finite dimensions by a vector $\boldsymbol{\theta}_{\text{ctrl}} \in \mathbb{R}^{d_c}$. The framework's reference decoder is a Gaussian-RBF basis: with $\{\varphi_a(t)\}_{a=1}^{d_c}$ a fixed basis on $[0, T]$, a logit bias c_U chosen so that $\boldsymbol{\theta}_{\text{ctrl}} = 0$ produces a canonical default U_{def}, and an upper bound $U_{\max}$,

$$U(t;\, \boldsymbol{\theta}_{\text{ctrl}}) \;=\; U_{\max} \cdot \text{sigmoid}\!\left(c_U + \sum_{a=1}^{d_c} (\boldsymbol{\theta}_{\text{ctrl}})_a\, \varphi_a(t)\right). \tag{20}$$

The sigmoid envelope keeps $U(t; \boldsymbol{\theta}_{\text{ctrl}}) \in [0, U_{\max}]$ for every $\boldsymbol{\theta}_{\text{ctrl}}$; the search is unconstrained in $\mathbb{R}^{d_c}$. Other decoders (piecewise constant, neural network, etc.) plug in via the same

`ControlSpec` interface; the framework only requires the decoder to be a JAX-traceable mapping $\mathbb{R}^{d_c} \to \mathbb{R}^{n_u \times T}$.

8.3 Tempered control posterior

Following the control-as-inference tradition (Kappen, 2005; Toussaint, 2009; Levine, 2018), the framework defines a probability measure on $\boldsymbol{\theta}_{\text{ctrl}}$ by treating the negative cost as an unnormalised log-likelihood. With inverse temperature $\beta > 0$ and a Gaussian prior $\boldsymbol{\theta}_{\text{ctrl}} \sim \mathcal{N}(0, \sigma^2_{\text{prior}} I_{d_c})$, the *control posterior* is

$$q_\beta(\boldsymbol{\theta}_{\text{ctrl}}) \;\propto\; \exp(-\beta\, J(\boldsymbol{\theta}_{\text{ctrl}}))\, \mathcal{N}(\boldsymbol{\theta}_{\text{ctrl}};\, 0,\, \sigma^2_{\text{prior}} I_{d_c}), \tag{21}$$

where $J(\boldsymbol{\theta}_{\text{ctrl}}) := J(U(\,\cdot\,; \boldsymbol{\theta}_{\text{ctrl}}))$. Sampling from q_β is performed by the SMC^2 machinery of Section 5, but with the unbiased filter likelihood $\widehat{L}_N(\boldsymbol{\theta}_{\text{dyn}})$ replaced by the cost-likelihood $\exp(-\beta\, J(\boldsymbol{\theta}_{\text{ctrl}}))$. The inverse temperature β is calibrated automatically from the cost spread of the prior cloud, and the tempered schedule $\lambda \in [0, 1]$ smoothly interpolates from prior to control posterior.

Proposition 8.1 (Posterior mean as a regularised cost minimiser)**.** Let q_β be the control posterior (21) and write $\bar{\theta} := \mathbb{E}_{q_\beta}[\boldsymbol{\theta}_{\text{ctrl}}]$. Then

$$\bar{\theta} \;=\; \operatorname*{argmin}_{\theta \in \mathbb{R}^{d_c}} \Big\{ \mathbb{E}_{q_\beta}\big[\|\theta - \boldsymbol{\theta}_{\text{ctrl}}\|_2^2\big] \Big\}, \tag{22}$$

i.e. the posterior mean is the L^2 minimiser of squared distance under the tempered measure. When J is approximately quadratic near its minimum and the prior is not too informative, $\bar{\theta}$ sits in a neighbourhood of the unregularised minimiser $\operatorname{argmin}_\theta J(\theta)$ of size $O(\beta^{-1/2})$.

The framework uses $\bar{\theta}$, computed as the equally-weighted mean of the SMC^2 parameter cloud at $\lambda = 1$, as the controller output. The corresponding schedule is then $U(\,\cdot\,; \bar{\theta})$ via (20).

8.4 Hard-indicator vs soft-sigmoid chance-constrained variants

Many practical control problems are most naturally specified via a *chance constraint* on a state event rather than via a smooth running cost. A typical form is

$$\mathbb{P}\big[X_t \in \mathcal{B}_{\text{bad}}(U_t)\big] \;\le\; \alpha, \qquad \forall\, t \in [0, T], \tag{23}$$

where $\mathcal{B}_{\text{bad}}$ is some user-defined "bad set" (e.g. the basin of attraction of a collapsed equilibrium). The framework supports two ways of encoding this constraint inside the control-posterior cost.

(a) Hard indicator — pure-IS particle weighting. Replace the running cost ℓ by the indicator $\ell^{\text{hard}}(X_t, U_t) = \mathbb{1}[X_t \in \mathcal{B}_{\text{bad}}(U_t)]$ and aggregate as a per-bin violation rate. Designed for pure-SMC^2 controllers that re-weight particles by their empirical violation rate at each tempering step. Indicator-based; not differentiable in $\boldsymbol{\theta}_{\text{ctrl}}$.

(b) Soft sigmoid surrogate — HMC compatible. Replace the indicator by a sigmoid surrogate

$$\ell^{\text{soft}}(X_t, U_t) \;=\; \text{sigmoid}\left(\frac{\beta_{\text{chance}}\,\big(d_{\text{bad}}(X_t, U_t)\big)}{\text{scale}} \right), \tag{24}$$

where $d_{\text{bad}}(X_t, U_t)$ is a signed distance (positive when X_t is in the bad set, negative otherwise). The scaling parameter β_{chance} controls the sharpness; as $\beta_{\text{chance}} \to \infty$ the surrogate converges to the hard indicator. Differentiable in $\boldsymbol{\theta}_{\text{ctrl}}$ via the trajectory; designed for HMC inner kernels.

Empirical comparison: soft works, hard does not. Both variants have been implemented and tested side-by-side on the chance-constrained control of a representative model. The soft variant, run with HMC inside the outer SMC^2 and a β annealed from ~ 50 at low λ to $\sim 10^4$ near $\lambda = 1$, converges to the correct chance-constrained optimum and produces schedules that respect the constraint at the prescribed rate. The hard variant — which matches the framework's importance-weighted SMC^2 structure more naturally, but provides no gradient information to drive HMC — does *not* converge to the same optimum in practice. The empirical recommendation is therefore:

Remark 8.1 (Use the soft variant)**.** For chance-constrained control problems, use the soft sigmoid surrogate (24) with HMC. The hard indicator formulation is preserved in `models/fsa_high_res/control_v5.py:evaluate_chance_constrained_cost_hard` (in the FSA model dev sandbox repository) for documentation / regression purposes only — it is not the production cost function, and should not be used to drive a deployed controller.

The detailed numerical comparison and the per-bin behaviour of both variants is in the FSA model documentation; that is where the empirical run lives. The framework-level point for this document is that both code paths exist, they target the same chance constraint (23), but only the soft path delivers a working controller in practice.

8.5 Implementation pointer

- `smc2fc/control/control_spec.py` — the `ControlSpec` dataclass: cost callable, schedule decoder, horizon length, initial state, reference parameters.
- `smc2fc/control/tempered_smc_loop.py` — the controller's outer loop. Built on the same SMC^2 engine as the inference side; the only difference is that the log-density it operates on is the cost-as-likelihood $-\beta\, J(\boldsymbol{\theta}_{\text{ctrl}})$ rather than $\log \widehat{L}_N(\boldsymbol{\theta}_{\text{dyn}})$.

9 Receding-horizon model-predictive control

This section closes the loop. The filter of Sections 3–7 produces a posterior over $\boldsymbol{\theta}_{\text{dyn}}$ on each observation window; the controller of Section 8 produces a posterior over $\boldsymbol{\theta}_{\text{ctrl}}$ given the current state estimate and the dynamics-parameter posterior. The MPC driver orchestrates the two: at each replan time, advance the plant under the previous stride's control, refresh the filter posterior on the new observation window, and call the controller for the next stride.

9.1 Finite-horizon stochastic optimal control

Consider a discrete-time stochastic system on a state $X_k \in \mathcal{X}$ and a control $U_k \in \mathcal{U}$, with transition kernel $X_{k+1} \sim p(\cdot \mid X_k, U_k)$, a stage cost $\ell : \mathcal{X} \times \mathcal{U} \to \mathbb{R}$, and a terminal cost $V_f : \mathcal{X} \to \mathbb{R}$. The *open-loop* finite-horizon problem of length H is

$$\min_{u_{0:H-1} \in \mathcal{U}^H} \mathbb{E}\left[\sum_{k=0}^{H-1} \ell(X_k, u_k) + V_f(X_H) \,\middle|\, X_0 = x \right], \tag{25}$$

with the expectation over the joint law of $X_{1:H}$ given the chosen sequence $u_{0:H-1}$. Denote its optimal value by $V_H(x)$ and a minimising sequence by $u^*_{0:H-1}(x)$. The mapping $x \mapsto u^*_0(x)$ is the open-loop optimal first-action policy.

9.2 The receding-horizon principle

In *receding-horizon* or *model-predictive* control, (25) is solved repeatedly on a sliding window. At each replan time t_n, given a current state estimate $\widehat{x}_n$:

1. Solve (25) for $u^*_{0:H-1}(\widehat{x}_n)$;
2. Apply only the first action (or the first stride of length $\Delta < H$) to the plant;
3. Observe new data, update the state estimate to $\widehat{x}_{n+1}$, return to step 1.

The closed-loop policy is a function of $\widehat{x}_n$ alone, so the controller is implementable. Stability of the closed loop is delicate. Mayne and Rawlings (2000) show that under suitable assumptions on the horizon length and the terminal cost / terminal set, the closed-loop state remains in a positively invariant set and converges to a neighbourhood of a desired equilibrium. The framework relies on the *long-horizon* variant of that theory: the planning horizon H is chosen long enough relative to the slowest time constant in the dynamics that the truncation error from neglecting trajectory beyond H is small.

9.3 Stochastic + output-feedback variants

Three flavours of stochastic MPC are commonly distinguished.

(i) *Certainty-equivalent* MPC. Replace the expectation in (25) by evaluation at a point estimate $\hat{X} = \mathbb{E}[X \mid \widehat{x}_n]$ and solve the resulting deterministic problem. Cheap, but discards uncertainty.

(ii) *Expected-cost* (stochastic) MPC. Keep the expectation, approximate it by Monte Carlo (sampling trajectories under the stochastic dynamics) or by scenario trees.

(iii) *Output-feedback* MPC. When X_k is not directly observed, the controller acts on a state estimate $\widehat{x}_n$ produced by an external estimator. For nonlinear non-Gaussian systems no *separation principle* holds in general; the joint design of estimator and controller is part of the modelling, not a free composition.

The `smc2fc` pipeline is in classes (ii) and (iii) simultaneously. The state is unobserved, so a state estimator is required: this is the rolling-window SMC^2 filter of Sections 5–7. The expectation in the cost (19) is kept, evaluated by Monte Carlo on the SDE under common random numbers, and minimised in $\boldsymbol{\theta}_{\text{ctrl}}$-space by the control SMC^2 of Section 8. The full closed- loop policy is the composition

$$(\text{observations}) \xrightarrow{\text{rolling SMC}^2\text{ filter}} \widehat{x}_n \xrightarrow{\text{control SMC}^2\text{, posterior mean}} \bar{\theta} \xrightarrow{(20)} U(\,\cdot\,;\bar{\theta}) \xrightarrow{\text{plant}} (\text{new observations}).$$

There is no separation principle here. The fact that this composition works at all is an empirical claim, validated on the model classes the framework has been tested against.

9.4 The closed-loop pipeline — generic algorithm

Algorithm 3 is the framework-level statement. Specific applications instantiate it via the bench-driver pattern of Section 9.5.

Algorithm 3 `smc2fc` closed-loop MPC (generic)

1: Initialise the plant; initialise an empty observation buffer.
2: **for** $n = 1, \ldots, N_w$ **do**
3: **Filter update.** If $n = 1$, run cold-start SMC^2 (Algorithm 2) on the current observation window to produce a posterior over $\boldsymbol{\theta}_{\text{dyn}}$; otherwise warm-start from the previous window's posterior via the Gaussian bridge (Section 7).
4: **State extraction.** For a small set of $\boldsymbol{\theta}_{\text{dyn}}$ particles drawn from the posterior, run the bootstrap filter (Algorithm 1) forward to the stride endpoint and compute the importance-weighted mean of the resulting state vector; call this $\widehat{x}_n$.
5: **Control plan.** Run the control SMC^2 of Section 8, conditioned on $\widehat{x}_n$ and the posterior-mean dynamics parameters $\bar{\theta}_{\text{dyn}}$, to obtain a posterior-mean schedule coefficient $\bar{\theta}_{\text{ctrl}}$.
6: **Apply.** Decode $U(\,\cdot\,; \bar{\theta}_{\text{ctrl}})$ via (20) and apply only its first stride (length Δ_s) to the plant; collect the resulting observations and append to the buffer.
7: **end for**

The horizon over which the controller plans (line 5) is the full remaining macrocycle, but only the first Δ_s bins of the plan are committed (line 6). This is the receding-horizon truncation of Section 9.2 applied to whatever model the framework is driving.

9.5 Driver pattern: plant, filter, controller

Application code instantiates Algorithm 3 via three artefacts.

(a) The plant interface, `StepwisePlant`. The "plant" is whatever object the controller acts upon to produce new observations. It is a user-supplied class with a single mutating method:

```
advance(stride_bins, U_schedule) → (trajectory, obs_dict)
```

A call to `advance` integrates the SDE for `stride_bins` time steps under the supplied schedule, samples the per-channel observations, and updates the plant's internal state. Pedagogically, `StepwisePlant` is the digital embodiment of the "real world" against which the controller is evaluated; mathematically, a single call samples one transition of the joint state-observation process under the chosen control over a stride of length Δ_s. The framework does not provide a generic `StepwisePlant`; each user model implements its own (the FSA-v5 model's `StepwisePlant` is one example).

(b) The MPC driver script. The receding-horizon orchestrator is a user-side script. Its execution sequence implements Algorithm 3 step by step: for each window n it

1. runs the rolling-window filter by calling `run_smc_window_native` (cold-start, $n = 1$) or `run_smc_window_bridge_native` (warm-start, $n \geq 2$), both from `smc2fc/core/jax_native_smc.py`;
2. extracts the posterior-mean dynamics parameters and the smoothed latent state $\widehat{x}_n$ at the end of the window;
3. assembles a `ControlSpec` (cost functional, horizon length, current state estimate, schedule decoder);
4. calls the control SMC^2 loop in `smc2fc/control/tempered_smc_loop.py`, which returns a posterior-mean schedule coefficient $\bar{\theta}_{\text{ctrl}}$ and a corresponding U schedule;
5. calls `plant.advance(stride_bins, U_schedule)` to apply only the first stride of that schedule;
6. appends the resulting observations to the data buffer and shifts the window.

(c) The posterior-mean handoff between windows. The smoothed state $\widehat{x}_n$ extracted in step 2 above is what initialises the next window's particle filter. Consecutive windows are therefore connected *at the state level* by a single point estimate (the importance-weighted smoothed mean) and *at the parameter level* by the Gaussian bridge of Section 7. This is the practical realisation of the warm-start that gives the rolling SMC^2 its computational tractability.

9.6 Recursive feasibility and soft barriers

State constraints in the cost (e.g. a fatigue ceiling, an admissible operating envelope) are enforced through *soft-barrier* penalties of the form $\lambda_b \cdot \max(g(X_t) - g_{\max}, 0)^2$ rather than through hard constraints. The choice is deliberate. Under stochastic dynamics with state-dependent diffusion and unbounded driving noise, an exact hard constraint $g(X_t) \leq g_{\max}$ cannot be guaranteed by any controller without making the admissible-control set empty for some achievable initial conditions; this is the standard *recursive feasibility* difficulty in stochastic MPC. A soft penalty preserves recursive feasibility — there is always a feasible schedule — and produces the constraint approximately, with the violation rate decreasing as λ_b is increased.

The framework's chance-constraint formulation of Section 8.4 is the more principled alternative when the "bad set" is precisely defined and a violation- rate budget is the natural specification. Soft barriers and chance constraints are not mutually exclusive; both can be combined in the same cost functional.

9.7 The flat-cost-surface argument

The controller commits the posterior mean $\bar{\theta}_{\text{ctrl}}$ rather than a sample from q_β or its full distribution. One might worry that this discards useful information, especially when the filter posterior over $\boldsymbol{\theta}_{\text{dyn}}$ is uncertain or biased. Empirically the policy is nevertheless robust to filter parameter drift, and the explanation is geometric: near the optimum, the cost $J(\boldsymbol{\theta}_{\text{ctrl}})$ is locally flat in the directions explored by the SMC^2 cloud. A first-order Taylor expansion at the posterior mean gives, for a perturbed estimate $\bar{\theta} + \delta$,

$$J(\bar{\theta} + \delta) \;\approx\; J(\bar{\theta}) + \tfrac{1}{2}\delta^\top H \delta, \qquad H = \nabla^2 J(\bar{\theta}), \tag{26}$$

so the additional cost from using $\bar{\theta}$ in place of an unknown "oracle" optimum scales with the Hessian. When H is small in the directions of greatest filter uncertainty, the closed-loop performance is correspondingly insensitive to filter accuracy. This is the mechanism by which a posterior-mean controller is reasonably robust even when the filter is rate-limited (e.g. early in the rolling schedule, before enough data has accumulated to identify the dynamics parameters tightly).

10 Configuration and knobs

This section is the configuration reference for `SMCConfig` in `smc2fc/core/config.py`. For each field: what it does mathematically, sensible default ranges, and the diagnostic signal that says "increase" or "decrease this". The fields are grouped into four blocks (outer SMC, inner PF, OT rescue, bridge) following the order in which they appear in the dataclass.

10.1 Outer SMC^2 (the parameter loop)

Field	Default	Meaning + tuning signal
`n_smc_particles`	256	Parameter cloud size M. Variance of posterior moments scales as $1/\sqrt{M}$. Increase if posterior CIs are too wide; decrease for faster runs.
`target_ess_frac`	0.5	ESS target for adaptive tempering. Lower (e.g. 0.3) = faster but riskier.
`num_mcmc_steps`	5	HMC moves per tempering level (cold start). Increase if posterior samples are correlated within a tempering level.
`max_lambda_inc`	0.05	Cap on $\delta\lambda$ per cold-start tempering step. Smaller $\Rightarrow$ more tempering levels but each one safer.
`hmc_step_size`	0.025	HMC leapfrog step size. Decrease if HMC acceptance is low; increase if HMC is wasting compute on tiny moves. Diagnostic: target acceptance rate ≈ 0.7.
`hmc_num_leapfrog`	8	Leapfrog steps per HMC trajectory. Higher = better mixing per move, more compute per move.

10.2 Inner particle filter

Field	Default	Meaning + tuning signal
`n_pf_particles`	400	State cloud size N inside the bootstrap filter. Variance of $\widehat{L}_N(\boldsymbol{\theta}_{\text{dyn}})$ scales as $1/\sqrt{N}$. Increase if $\widehat{L}_N$ is too noisy (HMC acceptance drops because of noisy gradients).
`bandwidth_scale`	1.0	Multiplicative scale on the Silverman bandwidth used by the Liu–West shrinkage of Section 3.3. Increase to 1.5–2.0 if you observe state-cloud collapse.

10.3 OT-rescue (Section 4)

Field	Default	Meaning + tuning signal
`ot_ess_frac`	0.05	ESS threshold for triggering OT-rescue. Below this fraction, the sigmoid-blend OT-resample fires. Leave at default unless you see sustained $N_{\text{eff}} < 5\%$ across many bins.
`ot_temperature`	5.0	Sharpness of the sigmoid blend in (9). Larger = sharper transition.
`ot_max_weight`	0.01	Cap on the OT contribution to the final particle cloud. Default 1% is conservative; raise to 0.05–0.10 if OT-rescue is firing but the cloud still degenerates.
`ot_rank`	5	Nyström low-rank approximation rank r. Higher = better kernel approximation, more compute. Tested up to 50.
`ot_n_iter`	2	Sinkhorn iterations. Default 2 is enough for the rescue role; raise to 5–10 for higher OT fidelity.
`ot_epsilon`	0.5	Gaussian-kernel bandwidth in OT. Smaller = sharper transport plan, more particle-collapse risk.

10.4 Bridge (Section 7)

Field	Default	Meaning + tuning signal
`num_mcmc_steps_bridge`	3	HMC moves per tempering level on warm-start (vs 5 for cold start). The bridge initialisation is closer to the new posterior than the prior, so fewer rejuvenation moves are needed per level.
`max_lambda_inc_bridge`	0.10	Cap on $\delta\lambda$ per warm-start tempering step (double the cold-start cap).
`bridge_type`	`"gaussian"`	One of `"gaussian"` (single Gaussian + LW shrinkage; the production default), `"mog"` (Gaussian mixture), `"schrodinger_follmer"` (the BW-geodesic variant; see Section 7's empirical assessment).
`bridge_mog_components`	2	Number of components when `bridge_type == "mog"`.
`sf_blend`	0.5	SF only: $t \in [0, 1]$ along the BW geodesic. 0 = previous posterior, 1 = new-posterior moment match.
`sf_entropy_reg`	0.0	SF only: entropic regularisation; 0 = exact Bures–Wasserstein.
`sf_q1_mode`	`"is"`	SF only: how to estimate the new-posterior Gaussian. `"is"` = single-step importance sampling. `"annealed"` = K-stage tempered SMC with random-walk Metropolis (more robust in high-D, more expensive).
`sf_annealed_n_stages`	3	Path B (annealed) only: number of intermediate stages.
`sf_annealed_n_mh_steps`	2	RW-MH moves per Path B stage.
`sf_use_q0_cov`	`False`	SF only: decoupled-mode flag — use the SF mean update but the previous-posterior covariance. Avoids over-inflation when the new-posterior covariance is MCMC-noisy.
`sf_info_aware`	`False`	SF only: per-eigenvector info-aware blending using the local Fisher information. Holds the previous-posterior mean in weakly-identified directions across windows. Off by default for bit-identical regression.
`sf_info_lambda_thresh_quantile`	0.5	SF only (when `sf_info_aware = True`): quantile of FIM eigenvalues used as the soft threshold between "well-identified" and "weakly identified".
`sf_info_blend_temperature`	1.0	SF only: τ in the sigmoid threshold. Smaller = sharper.

10.5 Other config dataclasses

`RollingConfig`. Window/stride/dt for the rolling- window framing. Fields: `window_days`, `stride_days`, `dt`, `n_substeps`, `max_windows`.

`MissingDataConfig`. An opinionated default model of missing-data corruption (rest days, random per-channel dropout, broken-watch gap). Useful for stress-testing the filter against realistic data quality issues. Fields: `dropout_rate`, `broken_watch_days`, `rest_days_per_week` plus the per-channel groupings `active_channels`, `passive_channels`, `all_obs_channels`.

10.6 Tuning workflow

In order, when a new application doesn't converge:

1. Diagnose first. Plot per-tempering-level ESS, HMC acceptance rate, $\widehat{L}_N$ variance across particles, and per-bin inner-PF ESS.
2. HMC acceptance below ~ 0.4? Reduce `hmc_step_size`.
3. HMC acceptance above ~ 0.9? Raise `hmc_step_size` (you're under-using each move).
4. Tempering bisection takes too many steps to find the right $\delta\lambda$? Lower `max_lambda_inc` (cap each step earlier).
5. Inner-PF ESS frequently dropping below 5%? Raise `n_pf_particles` first; if not enough, raise `ot_max_weight`.
6. Posterior moments still noisy after step 4? Raise `n_smc_particles`.

Do not change more than one knob at a time, and re-collect the diagnostics before deciding the next change.

11 Future work: optimising the soft-HMC controller

The chance-constrained variants of Section 8.4 settled the question of *which* cost surrogate to use: the soft sigmoid surrogate driven by HMC. This section is about the question that remains: how to make the soft-HMC controller faster without sacrificing the control quality that justified the choice in the first place. The hard-IS variant is not in scope here — it is a documented dead-end (Section 8.4, the "empirical comparison" remark) and is preserved only for regression testing.

11.1 The trade-off as observed today

Running the soft-HMC controller's tempered SMC^2 with high-quality settings — more leapfrog steps per HMC trajectory, more HMC moves per tempering level, finer tempering schedule — produces better control plans but costs more wall-clock time. This is, currently, the framework's main practical bottleneck. Planning latency scales roughly linearly with HMC work per tempering level:

$$\text{(wall-clock per plan)} \;\sim\; n_\lambda \cdot n_{\text{mcmc}} \cdot n_{\text{leap}} \cdot \text{(grad eval cost)},$$

where n_λ is the number of tempering levels, n_{mcmc} is the number of HMC moves per level, and n_{leap} is the leapfrog- step count per move. All three factors trade quality against speed, and the right operating point depends on the application's planning- latency budget. The remaining subsections collect the levers and the open directions for moving the Pareto frontier.

11.2 The tunable knobs and what each one costs

Four configuration fields are first-line levers. Their definitions and default values are in the configuration reference (Section 10); this subsection is about the quality–speed direction of each.

Knob	Quality direction	Speed direction	Diminishing-return knee
`hmc_num_leapfrog`	Higher = better mixing per trajectory; the proposal travels further per move.	Lower = fewer gradient evaluations per move.	Acceptance plateau around the standard $\tau \approx \pi/(2\omega)$ leapfrog tuning; raising past that wastes compute.
`num_mcmc_steps`	Higher = more rejuvenation per tempering level, lower autocorrelation between tempering levels.	Lower = fewer rejuvenation moves per level.	Per-level autocorrelation typically halves with each doubling up to about 5–8, then flattens.
`target_ess_frac`	Higher (closer to 1) = smaller $\delta\lambda$ per step, gentler tempering, more reliable convergence.	Lower = larger $\delta\lambda$, fewer tempering levels.	Below ~ 0.3 the bisection routinely fails to find a feasible step and the run aborts.
`max_lambda_inc`	Smaller = caps each tempering jump tighter, more steps but each one safer.	Larger = bisection allowed to take bigger leaps.	Larger than the empirical first-window mean of $\delta\lambda$ has no effect; smaller than ~ 0.01 produces redundant levels.

The four knobs interact. Lowering `target_ess_frac` to cut the tempering-level count typically requires raising `num_mcmc_steps` to compensate for the larger $\delta\lambda$ per step — otherwise the cloud cannot keep up with the temperature change and HMC acceptance collapses. The combinations that survive this trade are application-specific.

11.3 Open directions

The following are candidate avenues for improving the soft-HMC controller's Pareto frontier. None has been benchmarked; they are listed in roughly decreasing order of how cheaply they could be explored.

Profile first. Before changing any algorithmic knob, run `jax.profiler.start_trace` around a representative call to `smc2fc/control/tempered_smc_loop.py` and view the trace in TensorBoard. The wall-clock split between (i) HMC leapfrog gradient evaluations, (ii) the inner-PF likelihood, (iii) the on-device ESS bisection, and (iv) host–device transfer latency is the precondition for any informed knob change. The cheapest improvement may simply be removing a host sync that nobody noticed.

Adaptive HMC step-size schedule along λ. The existing diagonal-mass-matrix re-estimation (Section 5.4) tightens the geometry per tempering level, but the step size ϵ stays fixed. A schedule that starts with a larger ϵ at low λ (where the target is nearly the prior, broad and forgiving) and shrinks toward a target acceptance rate of ≈ 0.7 at high λ (where the posterior is tight and the geometry is most curved) could reduce the required n_{leap} at the easy end without sacrificing acceptance at the hard end. BlackJAX has a dual-averaging adapter that targets a fixed acceptance probability; wiring it into the JAX-native loop is straightforward.

Per-tempering-level adaptive leapfrog count. Currently `hmc_num_leapfrog` is a single integer applied at every tempering level. The optimal trajectory length depends on the local

geometry and on how far apart consecutive tempering levels are; a schedule that tracks the per-level $\delta\lambda$ chosen by the bisection (longer trajectories when $\delta\lambda$ is large) would let the controller spend its leapfrog budget where it matters.

Reuse the inference posterior as the controller's prior. The controller's tempered SMC^2 currently uses a fixed Gaussian prior $\boldsymbol{\theta}_{\text{ctrl}} \sim \mathcal{N}(0, \sigma^2 I)$ regardless of what the filter has learned about $\boldsymbol{\theta}_{\text{dyn}}$. When the filter posterior is informative — in particular, when its uncertainty has collapsed along the directions the controller cares about — starting from a prior that already encodes that information would shorten the tempering schedule. The technical question is: what is the right mapping from the filter's $\boldsymbol{\theta}_{\text{dyn}}$ posterior to the controller's $\boldsymbol{\theta}_{\text{ctrl}}$ prior? A heuristic such as "set σ^2_{prior} proportional to the filter's posterior uncertainty along each direction" is a starting point; a principled mapping would require some thought about the joint problem.

Per-bridge HMC budget tuning. `num_mcmc_steps_bridge` is currently 3 (vs 5 for cold start). The bridge initialisation (Section 7) is by construction close to the new posterior, so fewer rejuvenation moves should be sufficient. The open question is whether 3 is the right number or whether 1–2 would do, and whether the answer depends on the magnitude of the window-to-window posterior shift. A simple sweep on a representative multi-window benchmark would settle it.

Mixed-precision HMC. The HMC inner loop is dominated by matrix–vector products in the gradient evaluation; the outer SMC^2 ESS arithmetic and tempering bisection are not. Running HMC in `float32` and the rest of the pipeline in `float64` is a common $\sim 2\times$ speedup on GPU. The risk is that for ill-conditioned posterior geometries the lower precision degrades HMC acceptance enough to require more moves per level, eating the gain. Worth benchmarking.

Smaller controller particle count with re-warm-starts. The controller currently uses the same `n_smc_particles` as the inference side. The control posterior is typically lower-dimensional ($d_c \approx 8$ vs $d_\theta \approx 16$ for typical applications) and arguably needs fewer particles to characterise its mean adequately. A reduction to 128 or 96 controller particles, paired with the next window's bridge warm-start (Section 7 applied to the controller, not just to the filter), would scale the per-window controller cost down proportionally. This requires extending the bridge mechanism to the control posterior, which the current code does not do.

11.4 What's not on this list

A few things deliberately not in the future-work list:

- Replacing HMC with NUTS. Documented dead-end; see Section 6.4.
- Replacing the soft surrogate with the hard indicator. Documented dead-end; see Section 8.4.
- Replacing the Gaussian bridge with the SF/BW variant by default. Documented as not paying for itself; see Section 7.

These are not open questions. Future contributors should consult the relevant section before re-attempting.

A Code-API quick reference

A one-line description of every public function and class in the `smc2fc/` package, organised by sub-package. The intent is the "which file is X in?" lookup — not a substitute for reading the docstrings.

A.1 `smc2fc/core/`

`config.SMCConfig` Master configuration dataclass for outer SMC^2, inner PF, OT-rescue, and bridge. See Section 10.

`config.RollingConfig` Window/stride/dt configuration for rolling-window framing.

`config.MissingDataConfig` Default model of observation corruption (rest days, dropout, broken-watch gaps) used in stress tests.

`jax_native_smc.run_smc_window_native` Cold-start tempered SMC^2 for one observation window. JAX-native, compile-once. Production entry point for the first window.

`jax_native_smc.run_smc_window_bridge_native` Warm-start tempered SMC^2 for one observation window using the SF/BW bridge initialisation. Production entry point for windows $n \geq 2$ when `bridge_type = "schrodinger_follmer"`.

`jax_native_smc.run_tempered_chain` The on-device tempered loop itself (single-window, single `lax.while_loop`). Internal but exposed.

`jax_native_smc.TemperedState` `NamedTuple` carrying particles, log-weights, current λ, and HMC kernel state across iterations of the on-device loop.

`jax_native_smc_nuts.*` NUTS variant of the above. Documented dead-end; see Section 6.4. Not for production use.

`tempered_smc.run_smc_window` BlackJAX-wrapped cold-start equivalent of `run_smc_window_native`. Slower, easier to debug.

`tempered_smc.run_smc_window_bridge` BlackJAX- wrapped Gaussian-bridge warm-start. The `bridge_type = "gaussian"` path lives here; the JAX-native side currently only implements the SF variant.

`mass_matrix.estimate_mass_matrix` Diagonal mass- matrix estimator from a particle cloud (variance with a small regularisation floor). Re-called per tempering level.

`sampling.sample_from_prior` Convenience prior sampler in unconstrained space, used at cold start.

`sf_bridge.bures_wasserstein_map` BW transport matrix between two PSD covariances.

`sf_bridge.bw_geodesic` Closed-form Bures– Wasserstein geodesic between two Gaussians, parameterised by $t \in [0, 1]$.

`sf_bridge.estimate_target_gaussian` Single-step importance-sampling fit of q_1 (the new posterior) given q_0 (the previous posterior).

`sf_bridge.estimate_target_gaussian_annealed` K-stage tempered SMC fit of q_1 with random-walk Metropolis. Activated by `sf_q1_mode = "annealed"`.

`sf_bridge.fit_sf_base` Top-level entry point for the SF/BW bridge: returns the blended Gaussian to seed the next window.

`sf_bridge.estimate_fim_hessian` Diagonal Fisher- information estimate from the new-window log-likelihood Hessian; used by the info-aware blending mode.

`sf_bridge.per_eigenvector_blend` / `info_aware_bridge_mean` Per-direction blending between m_0 and m_1 keyed off the FIM eigenvalues. Activated by `sf_info_aware = True`.

A.2 smc2fc/filtering/

gk_dpf_v3_lite.make_gk_dpf_v3_lite_log_density_compileonce Build the closed-over inner-PF log-likelihood for a single observation window. The compile-once variant the JAX-native backend consumes.

gk_dpf_v3_lite.make_gk_dpf_v3_lite_log_density Per-call variant; closes over data values rather than just shapes. Used by the BlackJAX-wrapped backend.

_gk_kernel.compute_ess Effective sample size from log-weights.

_gk_kernel.silverman_bandwidth Silverman's rule bandwidth for the smoothing kernel.

_gk_kernel.log_kernel_matrix Log of the Gaussian kernel matrix between particles, used by the Liu–West smoothing step.

_gk_kernel.smooth_resample_basic Plain systematic resample with Liu–West shrinkage and Gaussian smoothing.

_gk_kernel.smooth_resample_ess_scaled As above, with bandwidth scaled by ESS to avoid over-smoothing when ESS is high.

_gk_kernel.smooth_resample Adaptive wrapper that dispatches to one of the variants above depending on ESS.

_gk_kernel.smooth_resample_ess_scaled_lw Log-weighted variant for use inside the bootstrap filter likelihood evaluation.

transport_kernel.compute_kernel_factor Nyström low-rank kernel factor between N particles and r random anchors; the K_{NR} matrix in Section 4.

transport_kernel.factor_matvec Mat-vec via the Nyström factorisation; $O(Nr)$ instead of $O(N^2)$.

transport_kernel.factor_matvec_batch Batched version of the same.

sinkhorn.sinkhorn_scalings Sinkhorn iterations on the low-rank-kernel-factored transport problem, returning the (u, v) scaling vectors.

project.barycentric_projection Barycentric projection of a source cloud onto a target cloud given Sinkhorn scalings (u, v) and the kernel factor.

resample.ot_resample_lr Top-level OT-rescue: Sinkhorn $\rightarrow$ projection $\rightarrow$ sigmoid blend with the systematic-resample output.

A.3 smc2fc/control/

control_spec.ControlSpec Dataclass describing the controller's task: cost callable, schedule decoder, horizon length, initial state estimate, reference dynamics parameters.

config.SMCControlConfig Configuration dataclass for the controller's tempered-SMC^2 loop (counterpart to `SMCConfig` on the inference side).

tempered_smc_loop.run_tempered_smc_loop_native JAX-native control SMC^2 loop: tempers the prior toward the cost-as-likelihood and returns the posterior cloud (mean = recommended $\bar{\theta}_{\text{ctrl}}$).

tempered_smc_loop.run_tempered_smc_loop BlackJAX-wrapped reference equivalent.

rbf_schedules.RBFSchedule Gaussian-RBF schedule decoder (20): maps a coefficient vector to a $U(t)$ trajectory.

calibration.calibrate_beta_max Choose the high- λ inverse temperature from the cost spread of the prior cloud.

`calibration.build_crn_noise_grids` Pre-sample common-random-number diffusion shocks for use across the tempering schedule.

`diagnostics.plot_cost_histogram` / `plot_schedule_comparison` / `plot_trajectories` / `print_smc_step` / `eval` Post-hoc inspection helpers; not on the run-time critical path.

`lqg.controller.LQGSpec` / `LQGController` / `build_lqg_open_loop_schedule` LQG baseline with a finite-horizon Riccati solver. Provides a sanity-check comparator to the tempered-SMC^2 controller.

`lqg.linearize.linearize_drift_at` Local linearisation of the SDE drift at an operating point; used by the LQG baseline.

`lqg.riccati.solve_riccati_backward` / `compute_lqr_gain` Backward Riccati integration and LQR gain extraction.

A.4 `smc2fc/simulator/`

`sde_model.StateSpec` Description of a single SDE state component (name, prior, bounds).

`sde_model.ChannelSpec` Description of a single observation channel (which states it sees, noise model).

`sde_model.SDEModel` The user-supplied model interface: drift, diffusion, observation kernel, parameter priors. Section 2 expands on this.

`sde_solver_diffrax.solve_deterministic_jax` Deterministic-mean trajectory via diffrax (used for warm- starts and diagnostics).

`sde_solver_diffrax.solve_sde_jax` Stochastic SDE solve via diffrax with optional control schedule.

`sde_observations.generate_all_channels` Sample per-channel observations from a trajectory.

A.5 `smc2fc/transforms/` and top-level

`transforms.unconstrained.build_transform_arrays` Build the bound/scale arrays for the unconstrained-space reparameterisation.

`transforms.unconstrained.constrained_to_unconstrained` / `unconstrained_to_constrained` Bijection between the user-facing constrained $\boldsymbol{\theta}_{\mathrm{dyn}}$ and the HMC-friendly unconstrained u.

`transforms.unconstrained.log_prior_unconstrained` Log-prior in unconstrained space (with Jacobian correction).

`transforms.unconstrained.split_theta` Slice an unconstrained vector into dynamics and control halves.

`estimation_model.EstimationModel` User-side inference interface (parameter list, bounds, log-prior, link to `SDEModel`).